

QUESTION 1

Code:

```
# Create Nile time series

nile_ts <- ts(Nile, start = 1871, frequency = 1)
# 5-year moving average

moving_avg <- stats::filter(nile_ts, rep(1/5, 5), sides = 2)
# Time values

t <- time(nile_ts)
# Linear trend

model <- lm(as.numeric(nile_ts) ~ t)
# Fitted trend

trend <- fitted(model)
# Detrended residuals

detrended <- as.numeric(nile_ts) - trend
# Create two plotting areas

par(mfrow = c(2, 1))

# Raw series + moving average

plot(
  t,
  as.numeric(nile_ts),
  xlab = "Year",
  ylab = "Annual Flow"
```

```
main = "Nile Flow and 5-Year Moving Average"
```

```
)
```

```
points(
```

```
  t,
```

```
  as.numeric(nile_ts),
```

```
  pch = 16
```

```
)
```

```
points(
```

```
  t,
```

```
  as.numeric(moving_avg),
```

```
  pch = 16
```

```
)
```

```
# Detrended residuals
```

```
plot(
```

```
  t,
```

```
  detrended,
```

```
  xlab = "Year",
```

```
  ylab = "Residual",
```

```
  main = "Detrended Nile Flow"
```

```
)
```

```
points(
```

```
  t,
```

```
  detrended,
```

```
  pch = 16
```

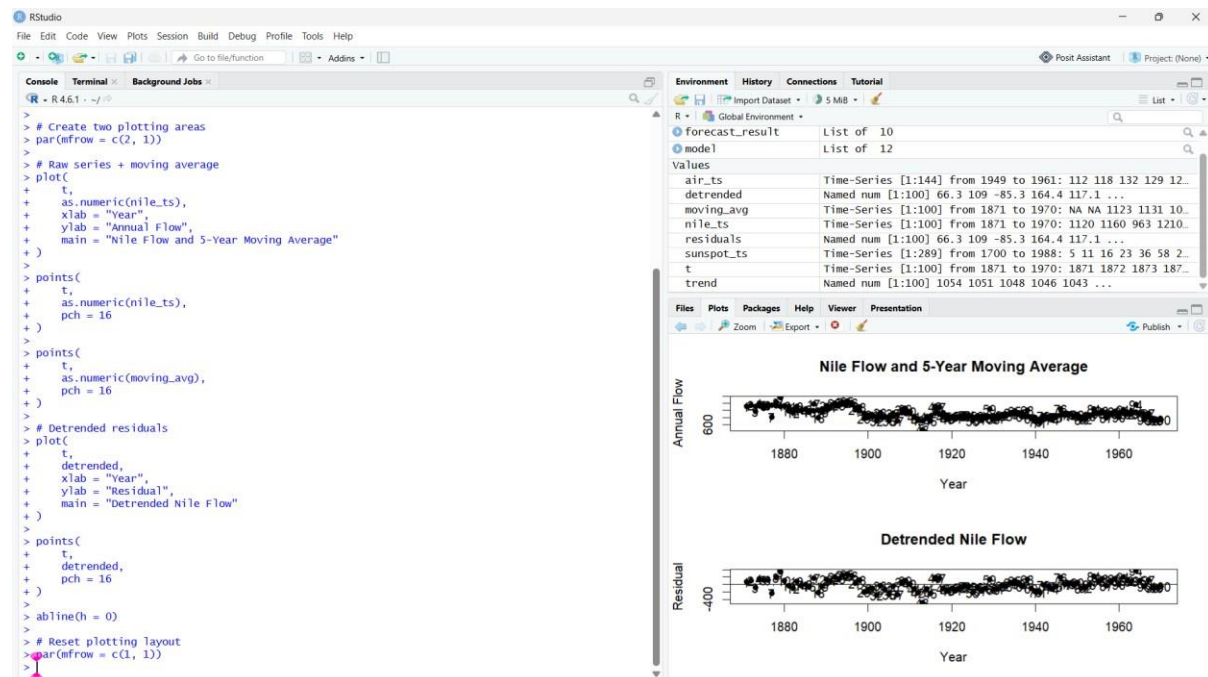
```
)
```

```
abline(h = 0)
```

```
# Reset plotting layout
```

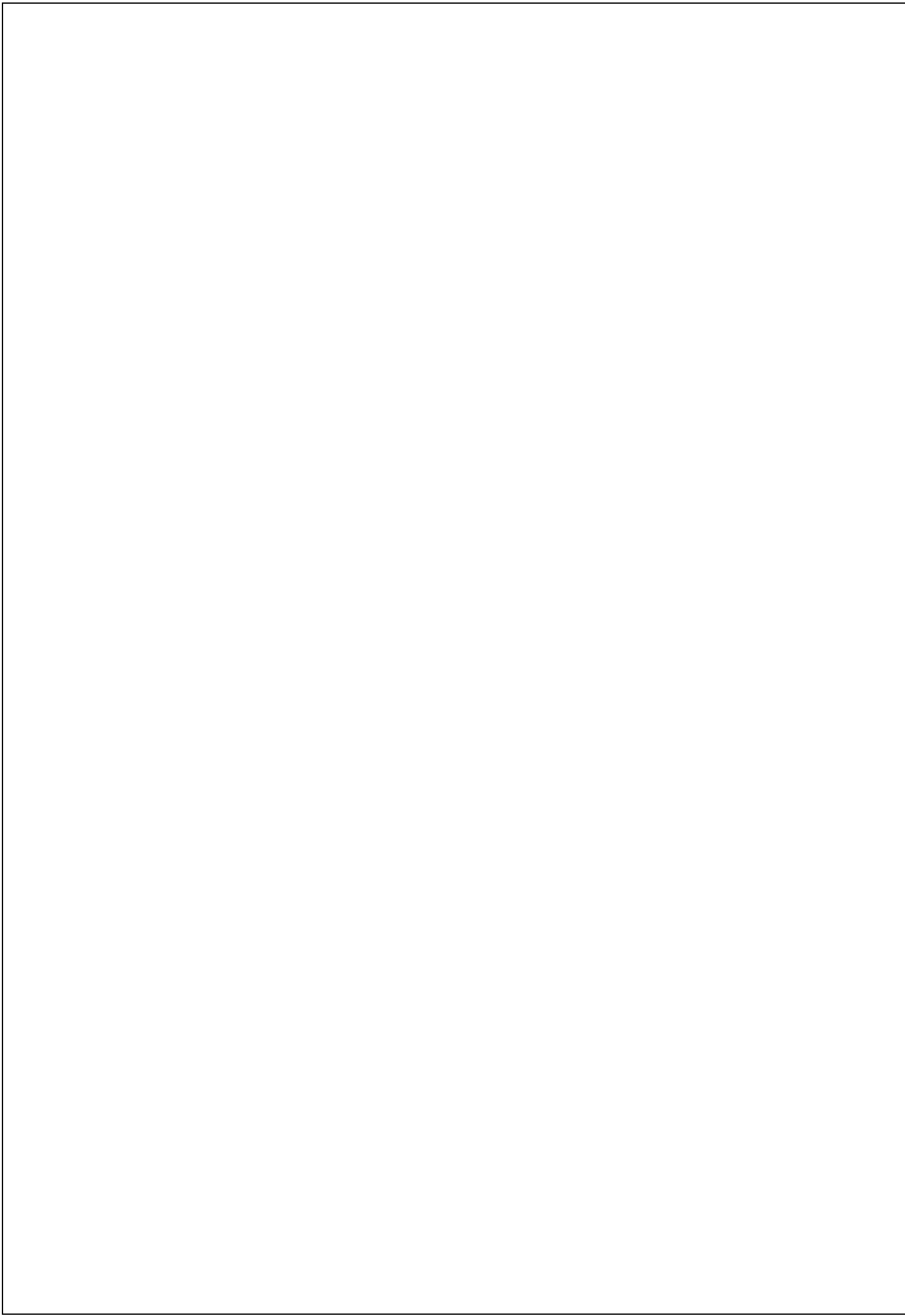
```
par(mfrow = c(1, 1))
```

Output:



Interpretation

- The **5-year moving average** smooths yearly fluctuations and makes the overall flow level easier to see.
- A noticeable **level shift occurs around the late 1890s**.
- The detrended residuals fluctuate considerably around zero, showing that the linear trend does not explain all variation.
- The residual pattern supports the presence of a **change in the underlying flow level**.
- Therefore, a **change-point or more flexible time-series model** would describe the data better than a single linear trend.



QUESTION 2

Code:

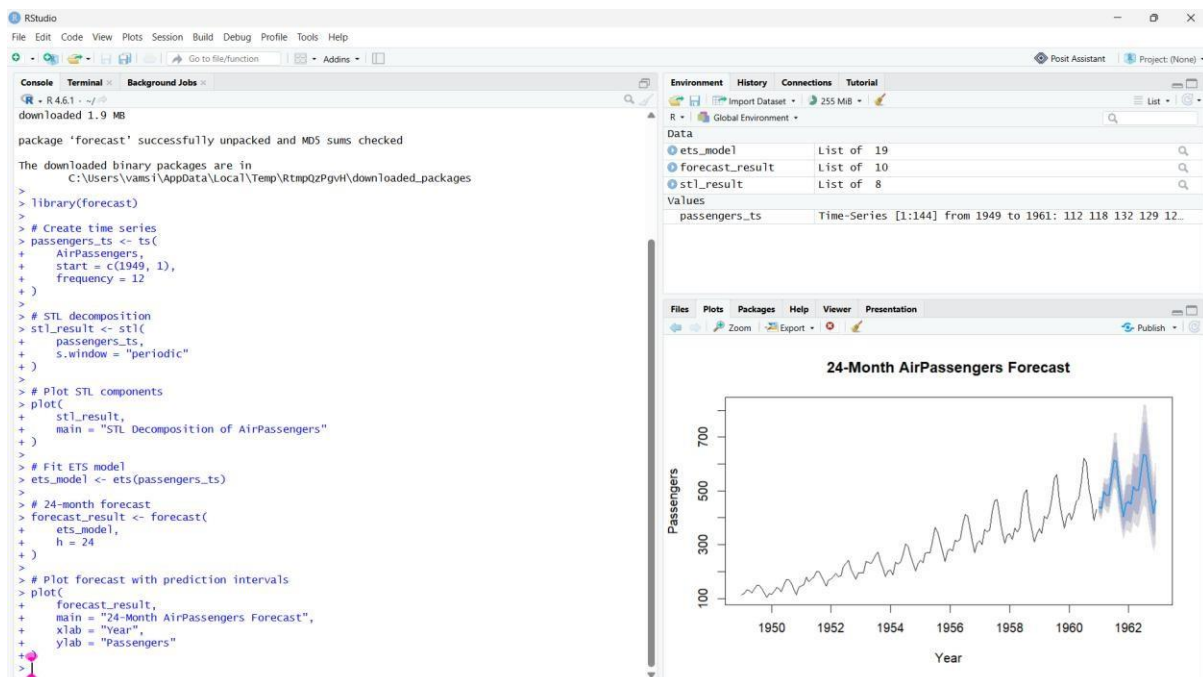
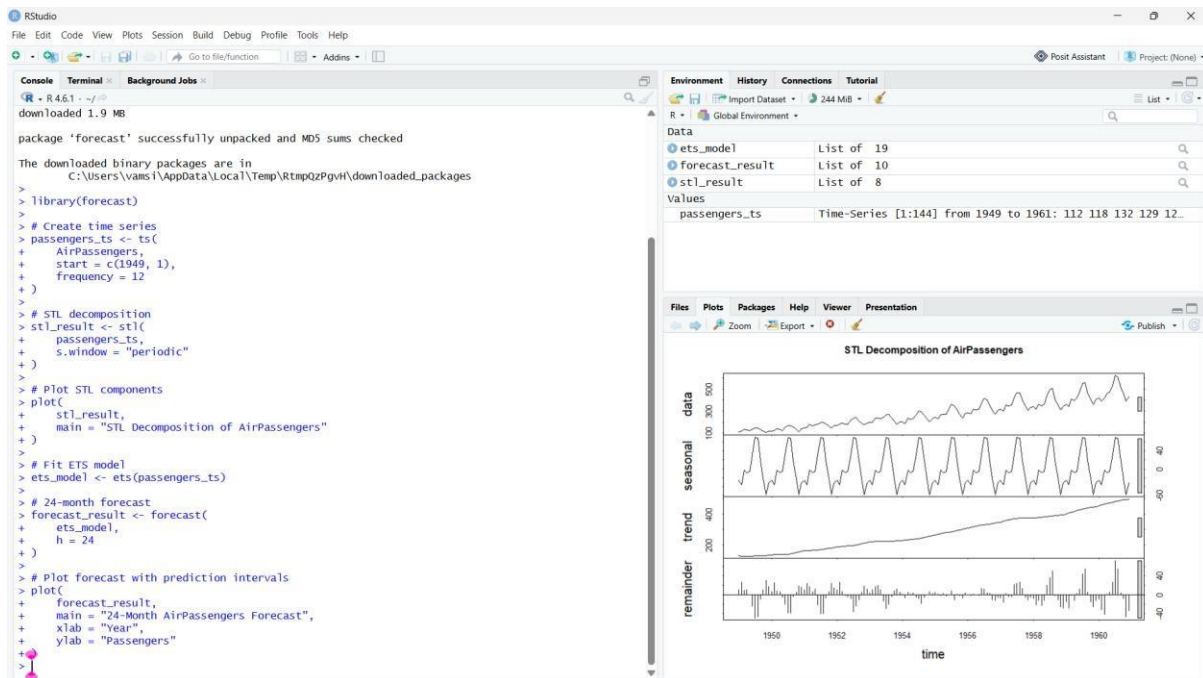
```
# Install forecast package once if needed
install.packages("forecast")

library(forecast)
# Create time series

passengers_ts <- ts(
  AirPassengers,
  start = c(1949, 1),
  frequency = 12
)
# STL decomposition
stl_result <- stl(
  passengers_ts,
  s.window = "periodic"
)
# Plot STL components
plot(
  stl_result,
  main = "STL Decomposition of AirPassengers"
)
# Fit ETS model
ets_model <- ets(passengers_ts)
```

```
# 24-month forecast
forecast_result <- forecast(
ets_model,
  h = 24
)
# Plot forecast with prediction intervals
plot(
  forecast_result,
  main = "24-Month AirPassengers Forecast",
  xlab = "Year",
  ylab = "Passengers"
)
```

Output:



Interpretation

- The **trend component** shows a strong long-term increase in passenger numbers.
- The **seasonal component** shows a clear repeating yearly pattern.
- Seasonal fluctuations become larger as the trend increases, so a **multiplicative**

decomposition is more appropriate.

- The ETS forecast continues both the increasing trend and the repeating seasonal pattern.
- The forecasted seasonal peaks and troughs are **consistent with historical seasonality**, while the prediction intervals widen over time.

QUESTION 3

Code:

```
# Install packages once if needed

install.packages("ggplot2")
install.packages("maps")
install.packages("dplyr")
install.packages("leaflet")

library(ggplot2)
library(maps)
library(dplyr)
library(leaflet)

# PART A: Choropleth Map

# Use built-in USArrests dataset
arrests <- as.data.frame(USArrests)
arrests$state <- tolower(rownames(arrests))

# Get US state boundaries
states <- map_data("state")

# Join data with map
map_data_us <- left_join(
  states,
  arrests,
```

```

    by = c("region" = "state")
)

# Choropleth map
ggplot(map_data_us, aes(x = long, y = lat, group = group, fill = Murder)) +
  geom_polygon(color = "white") +
  coord_fixed(1.3) +
  scale_fill_gradient(
    low = "lightyellow",
    high = "darkred"
  ) +
  labs(
    title = "US Murder Rate by State",
    fill = "Murder Rate",
    caption = "Source: USArrests dataset"
  ) +
  theme_void()

# State with highest murder rate
highest_state <- rownames(USArrests)[which.max(USArrests$Murder)]

cat("State with highest murder rate:", highest_state, "\n")
cat("Highest murder rate:", max(USArrests$Murder), "\n")

# PART B: Interactive Map
# Use built-in quakes dataset

```

```
quake_data <- quakes

# Color scale for depth
depth_colors <- colorNumeric(
  palette = "YlOrRd",
  domain = quake_data$depth
)

# Interactive earthquake map
leaflet(quake_data) %>%
  addTiles() %>%
  addCircleMarkers(
    lng = ~long,
    lat = ~lat,
    radius = ~mag * 2,
    color = ~depth_colors(depth),
    fillOpacity = 0.7,
    stroke = FALSE,
    popup = ~paste(
      "<b>Magnitude:</b>", mag,
      "<br><b>Depth:</b>", depth, "km"
    )
  ) %>%
  addLegend(
    "bottomright",
    pal = depth_colors,
    values = ~depth,
    title = "Depth (km)"
  )
```

Interpretation — Part A

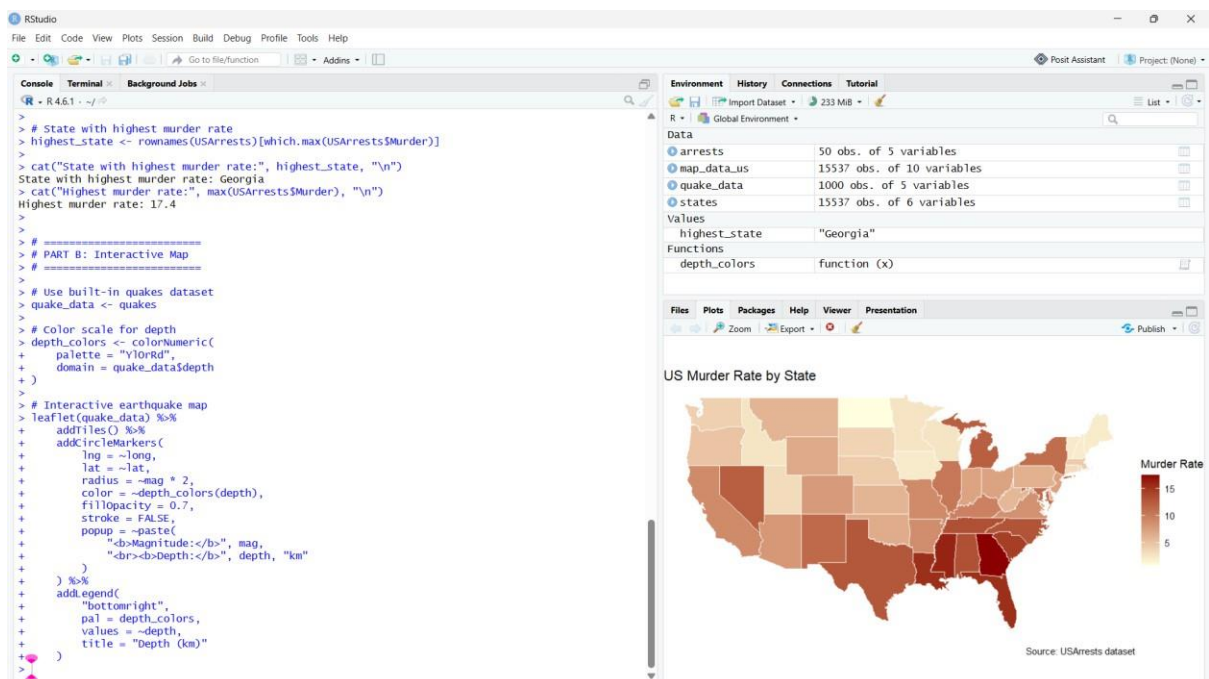
- The choropleth shows the **Murder rate across US states** using a sequential color scale.
- **Georgia** has the highest Murder rate in the built-in USArrests dataset.
- Higher murder rates appear more strongly concentrated in parts of the **South**.
- Darker regions represent states with higher murder rates.
- The map reveals a noticeable **regional clustering** rather than a completely random distribution.

Interpretation — Part B

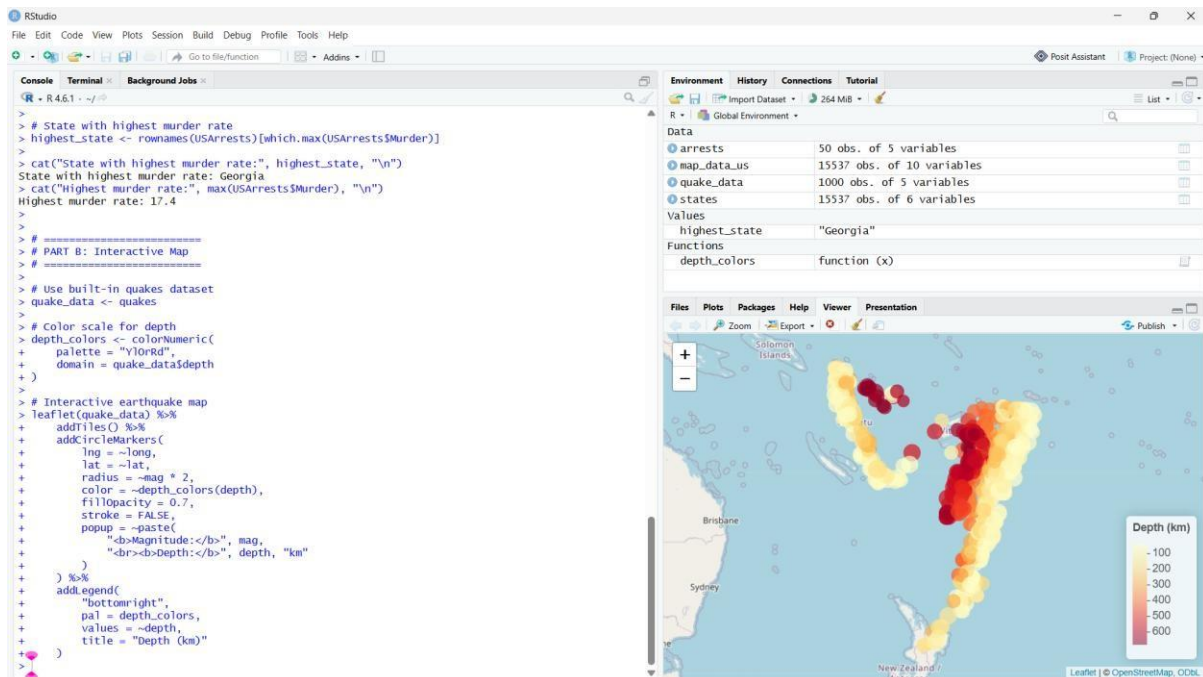
- Each circle represents an **earthquake location**.
- Larger circles indicate **higher-magnitude earthquakes**.
- Color represents earthquake **depth**, with the legend showing the depth scale.
- High-magnitude events are concentrated mainly along particular **tectonic regions**.
- The interactive popups display the **magnitude and depth** when a marker is clicked.

Output:

Part a — Choropleth Map



Part b — Interactive Geospatial Plot



QUESTION 4

Code:

```
# Install packages once if needed
```

```
install.packages("sf")
```

```
install.packages("maps")
```

```
install.packages("cartogram")
```

```
install.packages("ggplot2")
```

```
library(sf)
```

```
library(maps)
```

```
library(cartogram)
```

```
library(ggplot2)
```

```
# =====
```

```
# Load population data
```

```
# =====
```

```
data <- as.data.frame(state.x77)

data$state <- tolower(rownames(data))

# =====
# Get US state boundaries
# =====

us_map <- maps::map(
  "state",
  fill = TRUE,
  plot = FALSE
)

us_states <- st_as_sf(us_map)

us_states$state <- tolower(us_states$ID)

# Join population data
us_states <- merge(
  us_states,
  data,
  by = "state"
)

# =====
# Transform to Albers projection
# =====
```

```
albers_crs <- "+proj=aea +lat_1=29.5 +lat_2=45.5 +lat_0=23 +lon_0=-96"
```

```
us_states <- st_transform(  
  us_states,  
  crs = albers_crs  
)
```

```
# =====
```

```
# Standard map
```

```
# =====
```

```
standard_map <- ggplot(us_states) +  
  geom_sf(aes(fill = Population), color = "white") +  
  scale_fill_gradient(  
    low = "lightyellow",  
    high = "darkblue"  
  ) +  
  labs(  
    title = "Standard US Population Map",  
    fill = "Population"  
  ) +  
  theme_void()
```

```
# =====
```

```
# Population cartogram
```

```
# =====
```

```
cartogram_map <- cartogram_cont(  
  us_states,  
  weight = "Population",
```

```

    itermax = 5
)

cartogram_plot <- ggplot(cartogram_map) +
  geom_sf(aes(fill = Population), color = "white") +
  scale_fill_gradient(
    low = "lightyellow",
    high = "darkblue"
  ) +
  labs(
    title = "Population-Weighted Cartogram",
    fill = "Population"
  ) +
  theme_void()
# =====
# Display maps
# =====

print(standard_map)

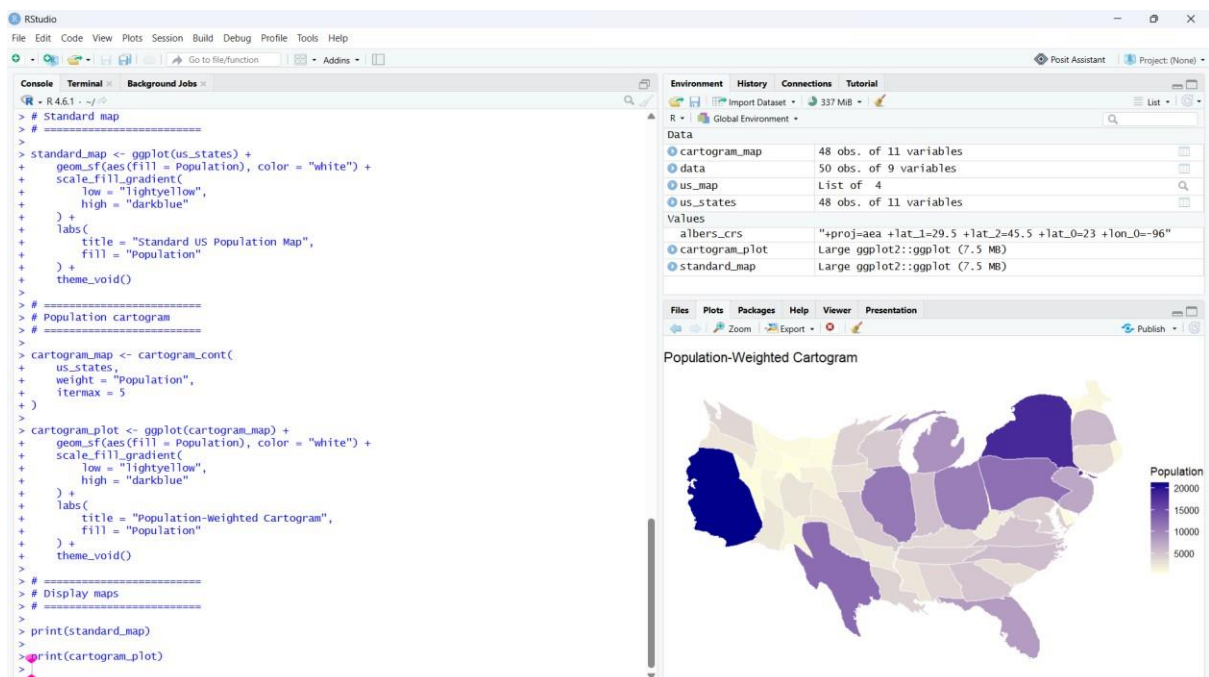
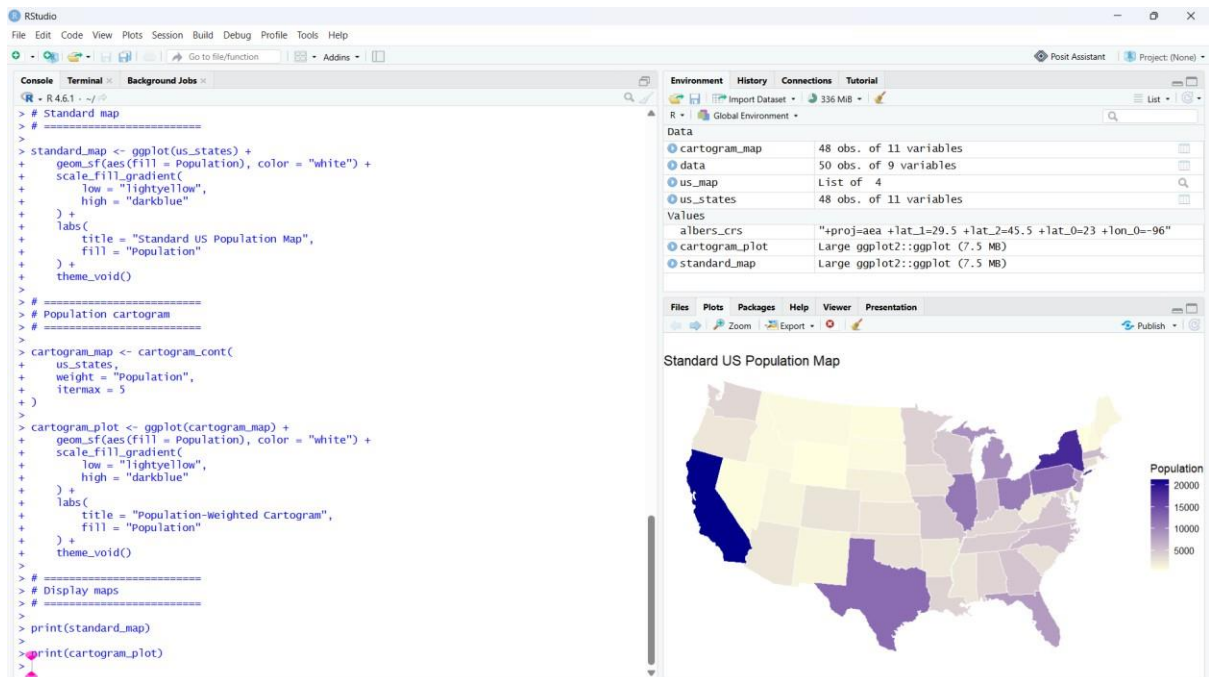
```

```
print(cartogram_plot)
```

Interpretation

- The standard map preserves the **actual geographic areas** of the states.
- The cartogram enlarges states with **higher populations** and shrinks less-populated states.
- Highly populated states such as **California and Texas** become visually much larger in the cartogram.
- The **Albers projection** provides a more suitable representation of US state areas than raw longitude/latitude coordinates.
- The cartogram emphasizes **population distribution**, while the standard map emphasizes **true geographic size**.

Output:



QUESTION 5

Code:

```
import pandas as pd

import plotly.graph_objects as go

import seaborn as sns

import matplotlib.pyplot as plt

import os

file = os.path.join(os.path.dirname(__file__), "volcano_elevation.csv")

data = pd.read_csv(file)

grid = data.pivot(index="y", columns="x", values="elevation")
# 3D Surface Plot

fig = go.Figure(data=[

    go.Surface(

        z=grid.values,

        x=grid.columns,

        y=grid.index

    )

])

fig.update_layout(

    title="3D Volcano Elevation Heatmap",

    scene=dict(

        xaxis_title="X",

        yaxis_title="Y",

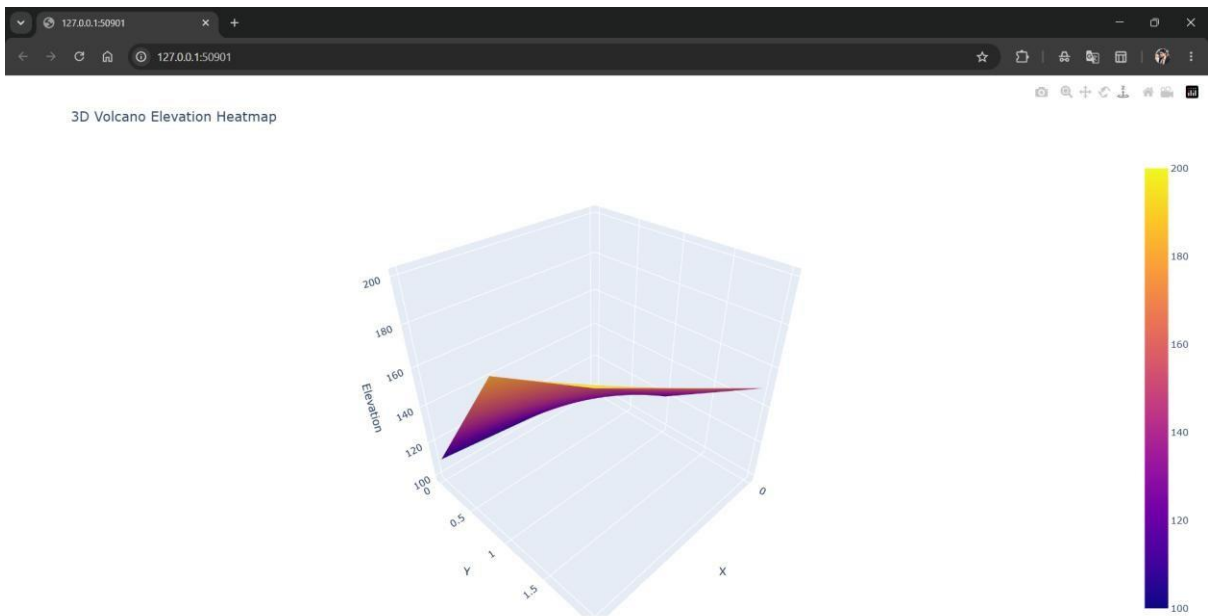
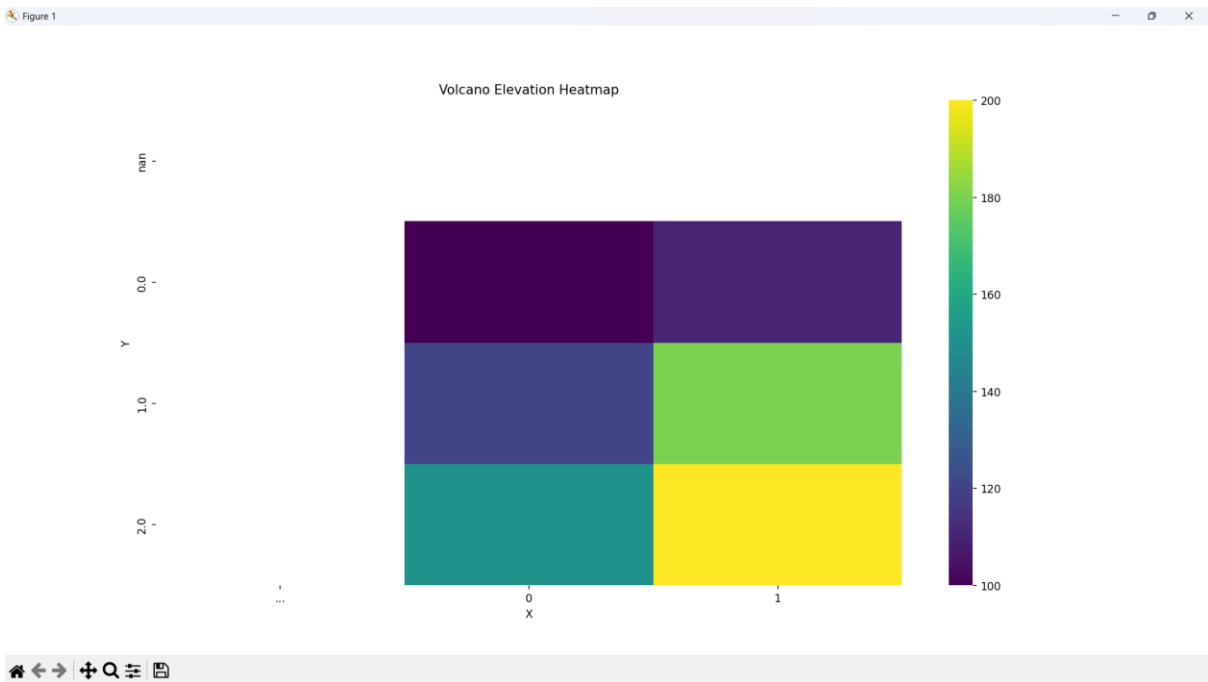
        zaxis_title="Elevation"

    )

)
```

```
)  
fig.show()  
# 2D Heatmap  
plt.figure(figsize=(8, 6))  
sns.heatmap(grid, cmap="viridis")  
plt.title("Volcano Elevation Heatmap")  
plt.xlabel("X")  
plt.ylabel("Y")  
plt.show()
```

Output:



Interpretation

The 3D surface plot and 2D heatmap visualize the **volcano elevation values** across different X and Y coordinates.

- **Higher peaks** in the 3D plot represent areas with **greater elevation**.

- **Lower regions** represent areas with **lower elevation**.
- The heatmap makes it easy to identify elevation patterns using different intensity levels.
- The volcano has a **central elevated region** surrounded by gradually decreasing elevations.